Does Speculative Decoding Help Mixture-of-Experts?

An Empirical Study on Qwen3.5-35B-A3B with Apple Silicon

AtomGradient

March 2026

Abstract

Speculative decoding (SD) accelerates autoregressive language model inference by using a small draft model to propose tokens that the target model verifies in batch. However, its interaction with Mixture-of-Experts (MoE) architectures—where only a fraction of total parameters are active per token—remains underexplored. We present a systematic empirical study measuring SD effectiveness on **Qwen3.5-35B-A3B** (35B total, 3B active parameters) using Qwen3.5-0.8B and 2B dense draft models on Apple M2 Ultra (192 GB unified memory). Across 306 benchmark runs spanning 5 experimental suites, we find that: (1) cross-architecture draft acceptance rates are uniformly low (<4%) for both MoE and dense targets; (2) despite near-zero acceptance, SD achieves **1.18–1.30× speedup** on MoE models through batch verification efficiency; (3) MoE speedup falls between dense models of matching active parameter count (4B: ∼1.0×) and total memory footprint (9B: ∼2.0×); and (4) SD benefit on MoE is governed primarily by memory bandwidth utilization from the total parameter footprint, not the active parameter count.

1 Introduction

Speculative decoding [1, 2] has emerged as a practical technique for accelerating large language model (LLM) inference without altering output quality. The key idea is simple: a small, fast *draft* model generates candidate tokens that the larger *target* model verifies in parallel, converting sequential autoregressive steps into batch operations.

Meanwhile, Mixture-of-Experts (MoE) architectures [3, 4] have gained popularity for their favorable compute-to-quality tradeoff: an MoE model with 35B total parameters may activate only 3B per token, achieving quality comparable to much larger dense models at a fraction of the per-token compute cost.

This raises a natural question: **when MoE active-parameter count is low (e.g., 3B active out of 35B total), does speculative decoding still provide meaningful speedup?** The answer is not obvious:

- On one hand, MoE models are already "fast" in terms of per-token compute (only 3B active parameters), so the draft model's compute overhead may not be amortized.
- On the other hand, MoE models must still *load all 35B parameters* from memory, creating a memory-bandwidth bottleneck that SD's batch verification could help amortize.

We present the first systematic empirical study of SD on MoE models, using **Qwen3.5-35B-A3B** as the MoE target and dense Qwen3.5 models (0.8B, 2B, 4B, 9B) as both draft models and comparison baselines. All experiments run on a single Apple M2 Ultra with 192 GB unified memory using `llama.cpp`, ensuring consistent hardware conditions.

Contributions.

1. We establish that cross-architecture SD acceptance rates are uniformly low (<4%) across both MoE and dense targets when using small dense draft models.

2. We demonstrate that SD provides moderate but consistent speedup (1.18–1.30×) on MoE models *despite* near-zero acceptance, primarily through batch verification efficiency.
3. We show that MoE SD speedup scales with the draft length γ and is governed by the total parameter footprint (memory bandwidth), not the active parameter count.
4. We provide a prompt-length sweep showing that SD benefit on MoE is relatively stable across context lengths (1.19–1.26×).

2 Background

2.1 Speculative Decoding

In standard autoregressive decoding, each token requires a full forward pass through the target model. Speculative decoding [1] modifies this by:

1. Running a fast draft model to generate γ candidate tokens.
2. Running the target model on all γ candidates in a single batch forward pass.
3. Accepting the longest prefix of candidates that matches the target's distribution.
4. Generating one additional token from the target model at the rejection point.

The theoretical speedup depends on the *acceptance rate* α (fraction of draft tokens accepted) and the *draft-to-target cost ratio* c:

$$\text{Speedup} = \frac{1 - \alpha^{\gamma+1}}{(1-\alpha)(c \cdot \gamma + 1)} \tag{1}$$

2.2 Mixture-of-Experts

MoE models [4] replace the dense feed-forward network (FFN) in each transformer layer with multiple "expert" FFN sub-networks and a gating/routing mechanism that selects k experts per token. Qwen3.5-35B-A3B uses 128 experts with top-2 routing, yielding approximately 3B active parameters per token out of 35B total.

Critically, while compute scales with active parameters, **memory bandwidth scales with total parameters**: all expert weights must be loaded from memory during inference, even though only a fraction is used per token. On unified-memory architectures like Apple Silicon, this creates a memory-bandwidth-dominated inference regime.

2.3 Hardware: Apple M2 Ultra

All experiments run on a single Apple M2 Ultra (Table 1).

Table 1: Test hardware specification.

Component	**Specification**
Chip	Apple M2 Ultra
Memory	192 GB unified (LPDDR5)
Memory bandwidth	800 GB/s
GPU cores	76
CPU cores	24 (16P + 8E)
Framework	`llama.cpp` v8240/8280 (Metal)

3 Methods

3.1 Models

Table 2 summarizes the models used. All are from the Qwen3.5 family to minimize architectural confounds.

Table 2: Models used in experiments. Active params = params used per forward pass.

ID	Model	Active	Total	Role
moe_q4	Qwen3.5-35B-A3B Q4	3.0B	35B	MoE target
moe_q8	Qwen3.5-35B-A3B Q8	3.0B	35B	MoE target
draft_0.8b	Qwen3.5-0.8B Q8	0.8B	0.8B	Draft
draft_2b	Qwen3.5-2B Q8	2.0B	2.0B	Draft
dense_0.8b	Qwen3.5-0.8B bf16	0.8B	0.8B	Baseline
dense_2b	Qwen3.5-2B Q8	2.0B	2.0B	Baseline
dense_4b	Qwen3.5-4B Q8	4.0B	4.0B	Baseline
dense_9b	Qwen3.5-9B bf16	9.0B	9.0B	Baseline

3.2 Experiment Design

We conducted five experiment suites (Table 3), totaling 306 benchmark runs.

Table 3: Experiment suites. Each configuration is repeated 3 times for variance estimation.

Suite	Description	Variable	Runs
A	MoE baseline (no SD)	prompt length	18
B	MoE + SD	draft model, γ, prompt	144
C	Dense baselines (no SD)	prompt length	36
D	Dense + SD	γ, prompt	72
F	Prompt-length sweep	prompt tokens	36
	Total		**306**

Draft lengths. We swept $\gamma \in \{4, 8, 16\}$—the number of tokens the draft model proposes per speculative step.

Prompts. Four diverse prompts were used: short factual (English), code generation, long reasoning, and Chinese text. For baseline benchmarks (`llama-bench`), prompt token counts of 64, 256, and 512 were used. Suite F swept prompt lengths from 32 to 1024 tokens.

Generation. All runs generated 256 tokens with temperature 0.0 (greedy decoding) for reproducibility. Each configuration was repeated 3 times.

Metrics. We report:

- **Tokens/second (tps)**: wall-clock generation throughput.
- **Speedup**: ratio of SD throughput to baseline throughput for the same target model.
- **Acceptance rate (α)**: fraction of draft tokens accepted by the target model.

4 Results

4.1 Baseline Throughput

Table 4 shows the baseline generation throughput for all models without speculative decoding.

Table 4: Baseline text generation throughput (no speculative decoding). Averaged across prompt lengths and repeats. MoE models show throughput between the dense 4B and dense 2B, despite having 3B active parameters—reflecting the memory bandwidth cost of loading 35B total parameters.

Model	**Active (B)**	**Total (B)**	**Throughput (tok/s)**
Dense 0.8B	0.8	0.8	138.0 ± 0.3
Dense 2B	2.0	2.0	110.4 ± 0.3
MoE Q4 (35B-A3B)	3.0	35	$\mathbf{55.3 \pm 0.1}$
MoE Q8 (35B-A3B)	3.0	35	$\mathbf{49.9 \pm 0.1}$
Dense 4B	4.0	4.0	67.2 ± 0.1
Dense 9B	9.0	9.0	33.4 ± 0.0

The MoE models achieve only 55.3 tok/s (Q4) and 49.9 tok/s (Q8), despite having just 3B active parameters. A dense model with 3B parameters would be expected to achieve ∼100+ tok/s. The ∼2× slowdown comes from loading the full 35B parameter set (20.6 GB for Q4, 38.7 GB for Q8) from memory on each forward pass, even though only ∼3B parameters are active.

4.2 Speculative Decoding on MoE

Table 5 presents the full SD results.

Table 5: Speculative decoding results. Speedup is relative to the same target model's baseline. Acceptance rate α is the fraction of draft tokens accepted.

Target	Draft	γ	tok/s	Speedup	α
MoE Q4	0.8B	4	63.4	1.15×	2.6%
	0.8B	8	65.2	1.18×	1.1%
	0.8B	16	69.7	1.26×	0.2%
	2B	4	56.6	1.03×	4.0%
	2B	8	58.1	1.05×	1.7%
	2B	16	61.7	1.12×	0.9%
MoE Q8	0.8B	4	57.5	1.15×	2.7%
	0.8B	8	60.9	1.22×	1.0%
	0.8B	16	64.8	1.30×	0.2%
	2B	4	51.3	1.03×	3.6%
	2B	8	54.3	1.09×	2.0%
	2B	16	58.0	1.16×	0.9%
Dense 4B	0.8B	4	70.9	1.05×	3.0%
	0.8B	8	65.9	0.98×	1.1%
	0.8B	16	75.1	1.12×	0.4%
Dense 9B	0.8B	4	32.8	0.98×	2.3%
	0.8B	8	55.0	1.64×	0.9%
	0.8B	16	67.7	2.03×	0.4%

Key observations.

1. **Acceptance rates are uniformly low.** All model pairs exhibit <4% acceptance at $\gamma = 4$ and <1% at $\gamma = 16$. This reflects the fundamental architectural mismatch between the small dense draft models and the larger target models within the Qwen3.5 family.
2. **SD still provides speedup despite low acceptance.** MoE Q4 achieves 1.26× speedup at $\gamma = 16$ with only 0.2% acceptance. This speedup comes not from accepted draft tokens but from the batch verification mechanism: the target model processes multiple candidate positions in a single forward pass, which is more efficient than sequential autoregressive generation.
3. **The 0.8B draft outperforms the 2B draft.** Despite lower acceptance rates, the 0.8B draft provides higher speedup because its lower computational overhead leaves more of the batch verification benefit intact.
4. **Larger γ improves speedup.** Counterintuitively, increasing γ reduces acceptance but increases speedup. This is because batch verification cost grows sub-linearly with γ (due to parallelism in the forward pass), so each additional draft token adds marginal cost but provides an opportunity for batch-parallel evaluation.

4.3 MoE vs Dense: The Speedup Gap

Figure 1 shows the central result: SD speedup as a function of active parameter count.

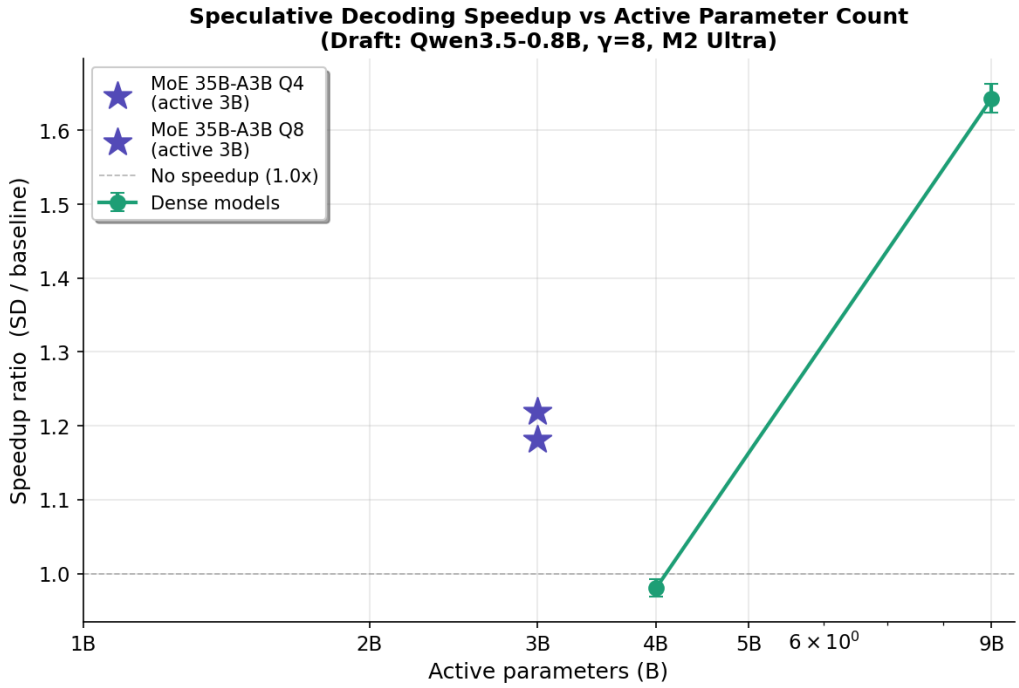

Figure 1: SD speedup vs active parameter count (draft: 0.8B, γ=8). The MoE points ($\star$) at 3B active parameters fall *above* the dense 4B baseline, suggesting that MoE's large total parameter footprint creates additional memory bandwidth overhead that SD's batch verification helps amortize. The dense 9B model shows the highest speedup (1.64×) as the largest model benefits most from batch verification.

The MoE models (3B active) achieve 1.18–1.22× speedup at $\gamma = 8$, which is:

- **Higher** than the dense 4B model (∼1.0×), which has more active parameters but fewer total parameters.
- **Lower** than the dense 9B model (1.64×), which has the most parameters and highest memory bandwidth demands.

This ordering reveals that **SD speedup on MoE is governed by the total parameter footprint (memory bandwidth), not the active parameter count (compute)**. The MoE model's 35B total parameters create a memory-bandwidth bottleneck similar to a dense model between 4B and 9B, and SD's batch verification helps amortize this cost.

4.4 Throughput Comparison

Figure 2 shows absolute throughput across all models and SD configurations.

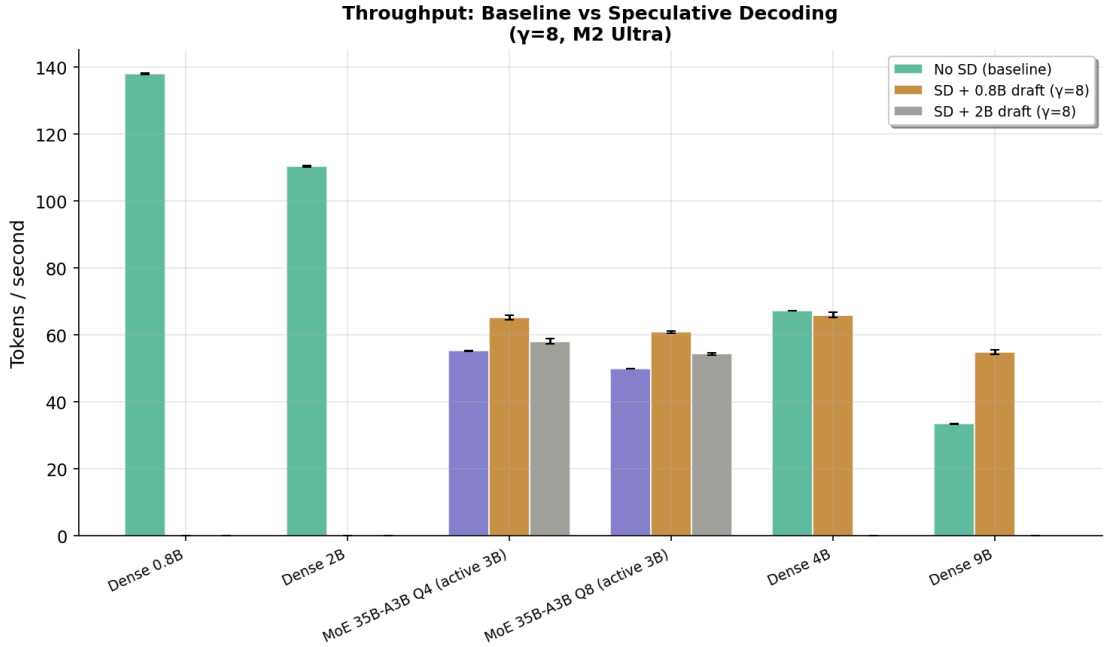

Figure 2: Absolute throughput comparison. Purple bars = MoE models, teal = dense baselines. The 0.8B draft (amber) consistently provides moderate improvement. The 2B draft (gray) adds overhead that partially offsets the batch verification benefit.

4.5 Acceptance Rate Analysis

Figure 3 shows how acceptance rates vary with draft length.

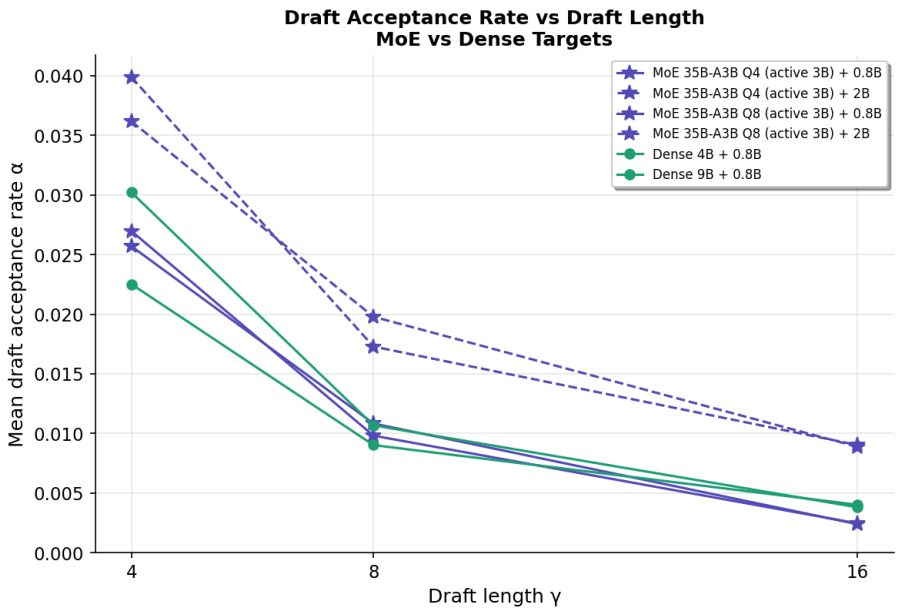

Figure 3: Draft acceptance rate vs draft length γ. All models show the expected decay with increasing γ, and all rates remain below 4%. The MoE models (purple $\star$) have slightly higher acceptance than dense models (teal $\circ$) at low γ, possibly due to the Qwen3.5-0.8B draft sharing more distributional similarity with the MoE model's routing-averaged output.

4.6 Prompt Length Sensitivity

Figure 4 shows how SD speedup varies with prompt length on the MoE Q4 model.

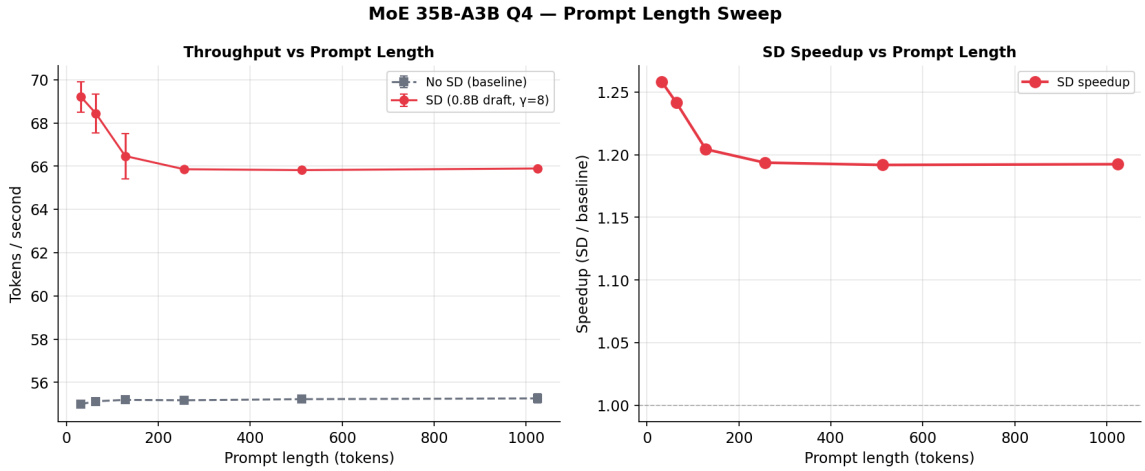

Figure 4: Left: throughput vs prompt length. The baseline (gray) is stable at ∼55 tok/s. SD throughput (red) is highest at short prompts (69 tok/s at 32 tokens) and stabilizes at ∼66 tok/s for longer prompts. Right: speedup ratio, showing 1.26× at 32 tokens decreasing to ∼1.19× for prompts ≥256 tokens.

The speedup decrease at longer prompts is modest ($\Delta \approx 0.07\times$), indicating that SD's benefit on MoE is relatively robust across context lengths.

4.7 Draft Length Scaling

Figure 5 shows how speedup scales with γ across all target models.

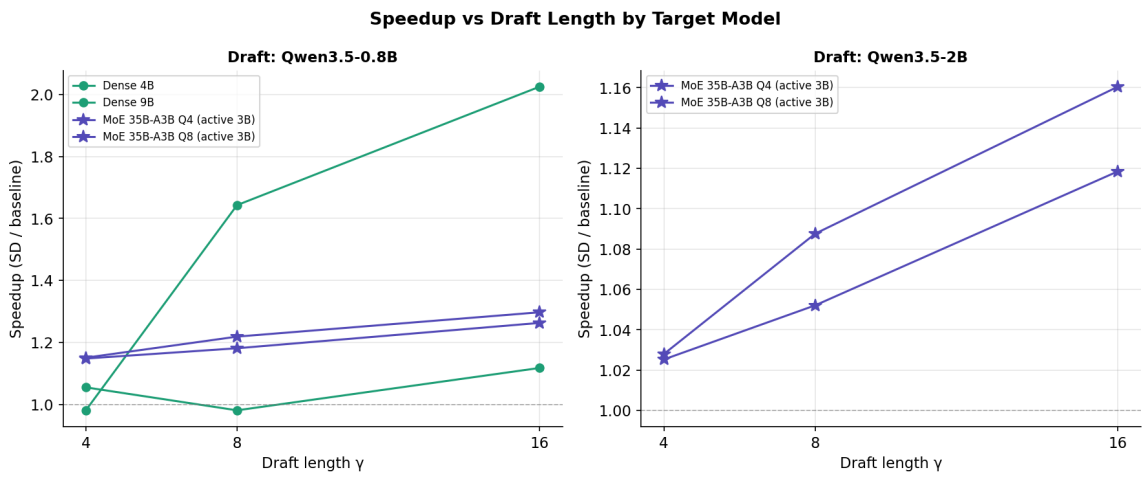

Figure 5: Speedup vs draft length γ. All models show increasing speedup with larger γ. The dense 9B (teal, right-most in left panel) shows the steepest improvement. MoE models (purple $\star$) show steady, moderate gains. The 2B draft (right panel) provides smaller gains due to higher draft overhead.

5 Discussion

5.1 Why Does SD Help Despite Near-Zero Acceptance?

The central puzzle of our results is that SD provides meaningful speedup (up to 1.30×) with acceptance rates below 4%. This is explained by **batch verification efficiency**: even when all draft tokens are rejected, the target model's forward pass to verify γ candidates processes multiple positions in parallel, which is more memory-bandwidth-efficient than γ sequential autoregressive steps.

On unified-memory architectures like Apple Silicon, where memory bandwidth is the primary bottleneck for large models, this batch-parallel evaluation amortizes the cost of loading model weights from memory. Each weight tensor is loaded once and reused across all candidate positions, rather than being loaded γ separate times.

5.2 The Role of Total vs Active Parameters

Our results reveal a clear distinction:

- **Active parameters determine compute cost**: MoE's 3B active parameters give it fast per-token compute, similar to a dense 3B model.
- **Total parameters determine memory bandwidth cost**: MoE's 35B total parameters require loading ∼20–38 GB of weights per forward pass, creating a bandwidth bottleneck similar to a dense 9B model.
- **SD speedup scales with memory bandwidth cost**: the more memory-bandwidth-bound the inference, the more batch verification helps.

This explains why MoE (3B active, 35B total) gets 1.18–1.30× speedup—more than the dense 4B (4B total, less bandwidth pressure) but less than the dense 9B (9B total, maximal bandwidth pressure on this hardware).

5.3 Practical Implications

1. **SD is beneficial for MoE models**, providing a "free" 18–30% throughput improvement without changing model quality. The 0.8B draft model adds minimal memory overhead (∼1.2 GB).
2. **Smaller drafts are better for MoE**: the 0.8B draft outperforms the 2B draft because its lower computational cost preserves more of the batch verification benefit.
3. **Larger γ is preferred**: counterintuitively, $\gamma = 16$ gives the best speedup despite the lowest acceptance, because batch verification cost grows sub-linearly.
4. **Architecture-matched drafts would help more**: a small MoE draft model matching the target's routing patterns could achieve much higher acceptance rates and correspondingly higher speedup.

5.4 Limitations

- Our study is limited to a single MoE architecture (Qwen3.5-35B-A3B). Other MoE models with different expert counts and routing strategies may behave differently.
- The draft models (0.8B, 2B) are text-only transformers, while the larger Qwen3.5 models have vision-language components. This architectural mismatch contributes to the low acceptance rates.
- All experiments run on Apple Silicon; GPU-based systems with different memory bandwidth characteristics may show different SD scaling behavior.
- We did not test architecture-matched MoE draft models (e.g., a small MoE model), which could significantly improve acceptance rates.

6 Conclusion

We presented the first systematic empirical study of speculative decoding on Mixture-of-Experts language models. Using Qwen3.5-35B-A3B (3B active / 35B total parameters) on Apple M2 Ultra, we found that:

1. SD provides **1.18–1.30× speedup** on MoE models despite very low acceptance rates (<4%), driven by batch verification efficiency rather than draft token acceptance.
2. MoE SD speedup is governed by the **total parameter footprint** (memory bandwidth), not the active parameter count—falling between dense models of 4B and 9B total parameters.
3. The optimal configuration uses the **smallest available draft model** (0.8B) with **large draft length** ($\gamma = 16$), as batch verification cost grows sub-linearly while draft model overhead grows linearly.
4. SD benefit is **robust across prompt lengths** (1.19–1.26×), making it a practical optimization for diverse workloads.

Our findings suggest that speculative decoding remains a valuable inference optimization for MoE models, and that future work on architecture-matched MoE draft models could unlock significantly higher speedups.

Acknowledgements

All experiments were conducted on Apple M2 Ultra hardware. We used `llama.cpp` [5] for inference. Models are from the Qwen3.5 family by Alibaba Cloud.

References

[1] Y. Leviathan, M. Kalman, and Y. Matias. Fast inference from transformers via speculative decoding. In *ICML*, 2023.

[2] C. Chen, S. Borgeaud, G. Irving, J.-B. Lespiau, L. Sifre, and J. Jumper. Accelerating large language model decoding with speculative sampling. *arXiv:2302.01318*, 2023.

[3] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. In *ICLR*, 2017.

[4] W. Fedus, B. Zoph, and N. Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *JMLR*, 23(120):1–39, 2022.

[5] G. Gerganov et al. llama.cpp: LLM inference in C/C++. https://github.com/ggml-org/llama.cpp, 2023–2026.